AI-DLC Quick Start

Author: Raja SP, AWS Labs
Source: https://github.com/awslabs/aidlc-workflows
Adaptation: Ricardo de Luna Galdino, EngSoft Learn
Download: inputs-quickstart.pdf

AI-DLC (AI-Driven Development Life Cycle) is a structured workflow that guides an AI assistant through planning, designing, and building software. Before starting a project, you provide two documents that tell the AI **what to build** and **what tools to use**.

What You Need to Provide

1. Vision Document -- what to build and why

Section	What to Write	How Long
Executive Summary	One paragraph: what is it, who is it for, why does it matter	3-5 sentences
Problem Statement	The specific business problem this solves	1-2 paragraphs
Target Users	Who will use it, what each user type needs	A table with one row per user type
Success Metrics	How you measure whether this project succeeded	A table with measurable targets
Full Scope Vision	Everything the product could become at maturity, organized by feature area	As many feature areas as needed
MVP Scope -- Features IN	Every feature included in the first release, with rationale	A table. If it is not listed, it is not in the MVP.
MVP Scope -- Features OUT	Features deliberately excluded from MVP, with reason and target phase	A table. This prevents scope creep.
Risks and Open Questions	What could go wrong, what is still undecided	Tables and bullet lists

Key principle: Separate the full vision from the MVP. The full vision is aspirational. The MVP is the smallest thing that delivers value.

Full guide: vision-document-guide.md
Worked example: example-vision-scientific-calculator-api.md

---

2. Technical Environment Document -- what tools to use

Section	What to Write	How Long
Languages	Required, permitted, and prohibited languages with versions	A table per category
Frameworks and Libraries	Required, preferred, and prohibited with rationale and alternatives	A table per category
Cloud Services	Allow list and disallow list of cloud services with constraints	A table per list
Architecture and Patterns	API style, data patterns, messaging, project structure	Short sections with tables
Security	Auth method, encryption, input validation, secrets management, and a chosen security compliance framework with controls documented per category	Several subsections
Testing	Test types, coverage targets, tooling, CI/CD gates	Tables
Example Code	Template code showing canonical patterns for endpoints, functions, tests, and infrastructure	Working code files in an `examples/` directory

Key principle: Be explicit about what is allowed and what is not. Allow lists and disallow lists prevent the AI from making assumptions.

Full guide: technical-environment-guide.md
Worked example: example-tech-env-scientific-calculator-api.md

---

Minimum Viable Input

If you want to start fast and fill in details later, provide at least this:

Vision (minimum)

```
1. One paragraph saying what you are building and for whom
2. A list of MVP features (what is IN scope)
3. A list of what is NOT in the MVP
4. Open questions -- things you already know are uncertain or unresolved
```

Open questions are optional but valuable. They feed directly into Requirements Analysis as pre-declared ambiguities, so AI-DLC addresses them early rather than surfacing them as surprises mid-design.

See example-minimal-vision-scientific-calculator-api.md for a worked example.

Technical Environment (minimum)

```
1. Language and version
2. Package manager
3. Web framework (if applicable)
4. Cloud provider and deployment model (or "local only")
5. Test framework
6. Prohibited libraries and services -- use a table: prohibited | reason | use instead
7. Security basics (auth method, input validation approach, secrets management)
8. Example code patterns -- one short example each for a typical endpoint, function, and tes
t
```

On item 6: including the reason and the recommended alternative is important. Without them, AI-DLC may honour the prohibition but not understand the intent well enough to make good substitution decisions.

On item 8: even one or two short examples give AI-DLC a concrete pattern to follow during code generation rather than inventing its own. This is the single highest-leverage addition beyond the basics.

See example-minimal-tech-env-scientific-calculator-api.md for a worked example of both.

Everything else can be answered through AI-DLC's clarifying questions during the Inception phase. The more you provide up front, the fewer questions the AI will need to ask.

Brownfield Projects

If you are adding to or modifying an existing codebase, your inputs need to answer a different set of questions. The full guides cover brownfield in detail, but the minimum is:

Vision (brownfield minimum)

```
1. Current state -- one paragraph describing what the system does today
2. What we are adding or changing -- a clear description of the change
3. Features IN scope for this iteration
4. Features OUT of scope for this iteration
5. What must NOT change -- existing components, APIs, or data the new work must not touch
6. Open questions
```

The "what must not change" section is critical. AI-DLC will run a Reverse Engineering stage to analyze your existing codebase, but being explicit about boundaries prevents it from proposing changes that would destabilize working parts of the system.

See example-minimal-vision-brownfield.md for a worked example.

Technical Environment (brownfield minimum)

```
1. Existing stack -- language, framework, database, infra -- with versions
2. What to add (new services, tables, components)
3. What must stay unchanged -- services, schemas, contracts, configs not to touch
4. Prohibited patterns -- libraries or approaches that conflict with the existing codebase
5. Security basics -- how auth and secrets work in the existing system
6. Example code patterns from the existing codebase
```

The example code patterns are especially important for brownfield. AI-DLC should generate code that looks like it belongs in the existing codebase, not code that introduces new conventions alongside old ones. Pull your examples from actual existing files.

See example-minimal-tech-env-brownfield.md for a worked example.

What Happens After You Provide These Documents

AI-DLC runs through two main phases:

Inception -- understand and plan

1. Detects your workspace (new project or existing code)
2. Analyzes requirements (asks clarifying questions if anything is unclear)
3. Creates user stories (if the project warrants them)
4. Builds an execution plan (which stages to run, which to skip)
5. Designs components and units of work (if complexity warrants it)

Construction -- design and build (per unit of work)

1. Functional design (business logic, domain models)
2. NFR requirements and design (performance, security, scalability)
3. Infrastructure design (maps to actual cloud services)
4. Code generation (writes the code, tests, and deployment artifacts)
5. Build and test (build instructions, test execution, verification)

Every stage requires your approval before proceeding. You can request changes, add skipped stages, or redirect at any gate.

File Overview

```
docs/writing-inputs/
  inputs-quickstart.md                                <-- You are here
  vision-document-guide.md                            <-- How to write a vision document
  technical-environment-guide.md                      <-- How to write a tech environment doc
ument

  -- Greenfield examples (new project from scratch) --
  example-vision-scientific-calculator-api.md         <-- Full example: CalcEngine vision
  example-tech-env-scientific-calculator-api.md       <-- Full example: CalcEngine tech env
  example-minimal-vision-scientific-calculator-api.md<-- Minimal example: CalcEngine vision
  example-minimal-tech-env-scientific-calculator-api.md<-- Minimal example: CalcEngine tech
env

  -- Brownfield examples (adding to an existing system) --
  example-minimal-vision-brownfield.md                <-- Minimal example: returns module on
existing platform
  example-minimal-tech-env-brownfield.md              <-- Minimal example: returns module on
existing platform
```